

AI-Powered Payment Gateway Platform

Overview

Modern payment platforms must support secure, low-latency, and scalable digital transactions across users, merchants, and financial institutions. Payment systems process millions of concurrent transactions while maintaining strict compliance, fraud prevention, and operational reliability. Support teams and merchants often struggle to quickly identify transaction failures, fraud signals, settlement issues, or processing bottlenecks. Build an AI-powered payment gateway platform capable of processing payments, supporting merchant integrations, analysing failures using NLP/LLM capabilities, and enabling intelligent operational workflows using Agentic AI.

Sample queries your system should handle:

- A merchant reports that 15% of card payments are failing this afternoon — what is the root cause and how do we fix it?
- A customer's international transfer was declined despite sufficient funds — what triggered the rejection?
- Detect and flag a series of micro-transactions that may indicate card-testing fraud.
- Generate a plain-English settlement summary explaining why a merchant payout differs from the expected amount.

Requirement 1 (Basic)

- User account management with sign-up, login, MFA, and wallet management.
- Payment processing for peer-to-peer and merchant transactions with multi-currency support.
- Transaction history, refunds, and dispute resolution workflows.
- NLP-enabled transaction failure reason detection — support agents can query failed transactions in natural language to retrieve structured failure reasons and suggested fixes.
- RAG-enabled support assistant for payment troubleshooting using historical transaction data.
- Hybrid search combining keyword and semantic retrieval over transaction records.
- Fraud detection and basic risk scoring with flagging of suspicious transactions.
- REST/gRPC APIs for merchant integrations with input validation guardrails and idempotency enforcement.
- Tool-calling support for payment status lookup and fraud analysis.
- Memory and session management for merchant support interactions.
- Notifications for payment confirmations and suspicious activity alerts.
- Simple ticket routing for operational escalations.
- Simple front-end interface for user and merchant interaction.

Requirement 2 (Advanced)

- Multi-agent orchestration for fraud detection, settlement reconciliation, and dispute handling.
- LLM-as-judge for validating dispute resolution workflows and ranking failure root-cause hypotheses with actionable resolution steps.
- DeepEval or equivalent evaluation framework for AI response quality; custom metrics for fraud prediction accuracy and payment failure prediction.

- Reranking of historical incidents based on transaction recency and resolution effectiveness.
- Token optimisation for bulk transaction analysis using chunking and summarisation strategies.
- Agent-to-agent communication between fraud, compliance, and settlement services for collaborative root cause analysis.
- Feedback loop for continuous learning and fraud pattern updates.
- Predictive analytics dashboard with settlement forecasting, chargeback probability scores, and real-time anomaly alerts.
- Integration with at least one real sandbox payment provider.
- Compliance automation for AML/KYC re-verification triggers and GDPR data-subject request handling.

Non-Functional Requirements

NFR Dimension	Target / Requirement
Performance	Transaction processing end-to-end latency < 2 seconds (P99)
Scalability	100,000 concurrent transactions/sec; 100 million registered users
Availability	99.999% uptime; automatic failover across regions
Security	End-to-end encryption; tokenisation of card data; fraud prevention pipeline
Compliance	PCI-DSS Level 1; GDPR; AML/KYC regulatory checks
Reliability	Atomic transactions; idempotent APIs; zero double-charge guarantee
Observability	Real-time monitoring; full audit logs; automated alerting
Geo Distribution	Multi-region active-active deployment; data residency compliance
B2B / B2C	Consumer wallet flows and merchant/enterprise API integrations

Dataset

Dataset Name: Synthetic Payment Gateway Transaction Dataset

Format: CSV / JSON

Participants must create and use their own synthetic dataset simulating payment transactions, fraud scenarios, disputes, settlements, and operational logs. A minimum of 10,000 records is recommended. Tools such as Python Faker or GPT-based generation scripts may be used.

Key Fields (minimum required schema):

Field Name	Type	Description
transaction_id	UUID	Unique identifier for each transaction
user_id	UUID	Anonymised user/sender account reference
merchant_id	string	Merchant identifier (null for P2P)
amount	float	Transaction amount in source currency
currency	string	ISO 4217 currency code (e.g. USD, EUR, INR)

payment_method	enum	card bank_transfer wallet upi
status	enum	success failed pending flagged reversed
failure_reason	string	Free-text description of failure cause
fraud_score	float	Risk score 0.0-1.0 assigned by fraud model
timestamp	datetime	ISO 8601 transaction timestamp
chargeback_flag	boolean	True if a chargeback was raised
settlement_status	enum	settled pending disputed reversed
resolution_notes	string	Free-text notes on how the issue was resolved
kyc_status	enum	verified pending failed
country_sender	string	ISO 3166 country code of sender
country_receiver	string	ISO 3166 country code of receiver

You may extend this schema with additional fields relevant to your implementation. Document all extensions in your README.

Deliverables

1. Architecture Diagram

Include a high-level view of your system in JPEG or PDF. Show how data is processed from the synthetic dataset through ingestion, AI/ML layers, APIs, and the user-facing interface. Label all major components, data stores, and integration points.

2. Design

Articulate the system design decisions taken and the trade-offs made (e.g. choice of vector store, message broker, caching strategy, AI model selection, synchronous vs. asynchronous processing).

3. Full Executable Code (Microservice)

Focus on clarity and modularity. Provide a README explaining: (a) project setup — how to install dependencies and run the service; (b) sample usage — at least one end-to-end example showing a test input and the resulting output.

4. Panel Presentation (10 minutes)

Demo of your working solution outlining the design and approach (8 minutes). Q&A; (2 minutes).